SYSTEM DESIGN · OPERATOR LAB

Oracle RAC 19c on Kubernetes with Oracle's Database Operator

How a two-node RAC database runs on a three-node kubeadm cluster in VirtualBox: the components, the networks, the shared storage, what the operator does, and the order in which it all comes up.

---

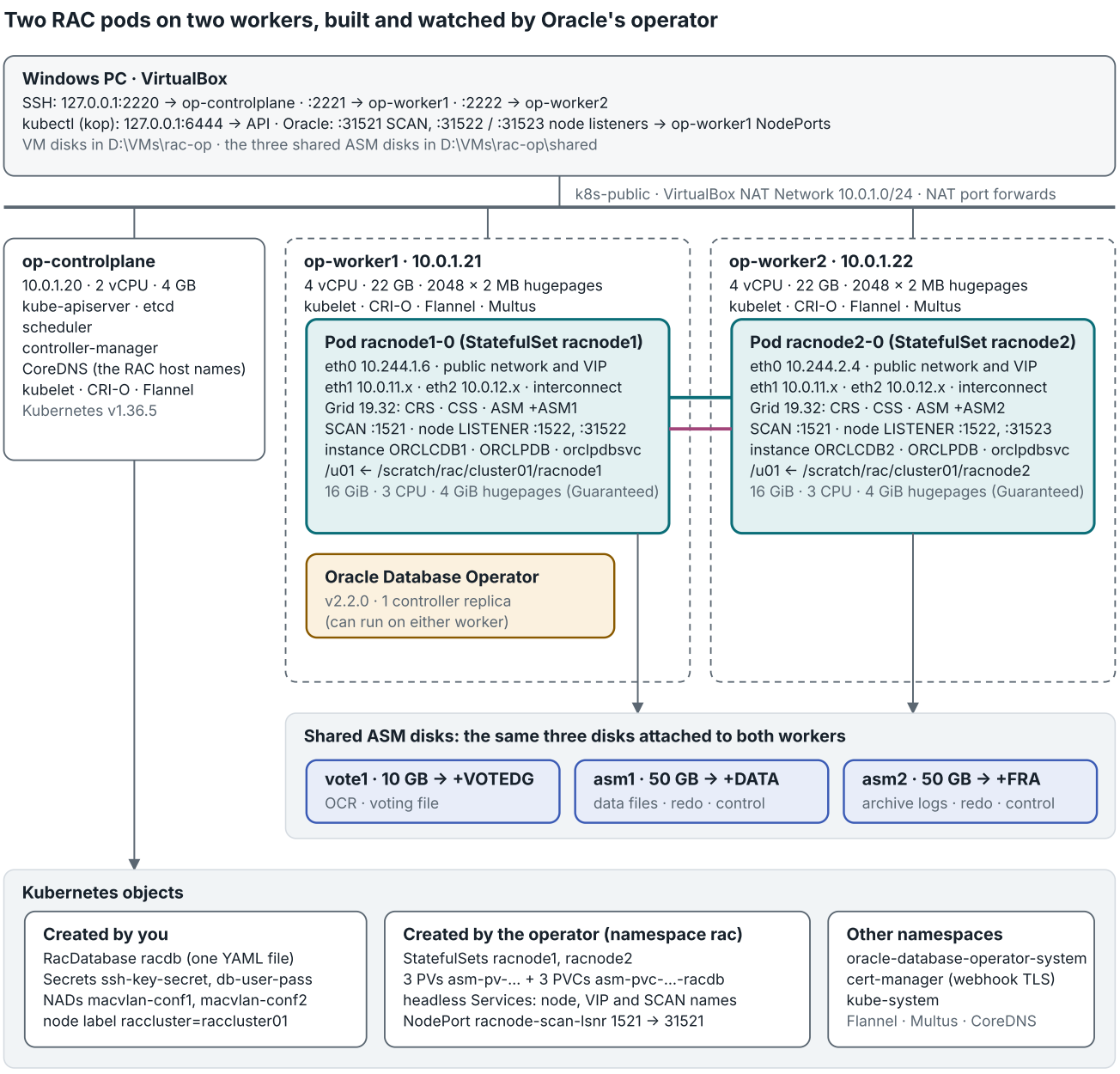
1 What this system is

A two-node **Oracle RAC 19c** database (Release Update 19.32) running in two Kubernetes pods. The Kubernetes cluster is built with **kubeadm** on three VirtualBox VMs on a single Windows PC. **Oracle's Database Operator** creates and watches the database: you describe the database in one `RacDatabase` YAML file, and the operator turns it into the pods, volumes and network names, then lets Oracle's own setup scripts inside the pods install Grid Infrastructure and create the database.

ITEM	VALUE
Cluster	kubeadm, Kubernetes v1.36.5 · CRI-O · Flannel + Multus · cluster name `rac-op`
Nodes	`op-controlplane` 10.0.1.20 (2 vCPU, 4 GB) · `op-worker1` 10.0.1.21 and `op-worker2` 10.0.1.22 (4 vCPU, 22 GB, 2048 hugepages each)
Operator	Oracle Database Operator v2.2.0, namespace-scoped (`oracle-database-operator-system` + `rac`), 1 controller replica, cert-manager v1.20.2
Image	`localhost/oracle/database-rac:19.3.0-slim`, built on each worker (Oracle Linux 9 base, 443 MB)
Software	Grid Infrastructure and Database 19.32 gold-image zips, staged on each worker
Database	`ORCLCDB` (instances `ORCLCDB1`, `ORCLCDB2`) · PDB `ORCLPDB` · service `orclpdbsvc` · ARCHIVELOG
Storage	three shared disks: `+VOTEDG` 10 GB, `+DATA` 50 GB, `+FRA` 50 GB, all EXTERNAL redundancy
Pods	`racnode1-0`, `racnode2-0`: 16 GiB memory, 3 CPU, 4 GiB hugepages each; SGA 3 GB, PGA 1 GB

Read the diagrams top-down: the whole system first, then the three things RAC needs from any platform (networks, shared storage, a way to build the nodes), then the order in which it all happened.

2 The whole system



The whole lab. Everything below the Windows box runs in three VirtualBox VMs on one Windows PC; the dashed boxes are the two worker VMs.

Layer by layer:

- **Windows PC.** Hosts VirtualBox and is the only way in: SSH and the Kubernetes API reach the VMs through **port forwards** on the VirtualBox NAT Network (127.0.0.1:2220–2222 for SSH, :6444 for kubectl). The VM disks, including the three shared ASM disks, are files on D:.
- **op-controlplane.** Runs the Kubernetes control plane: the **API server** (every change goes through it), **etcd** (where all objects are stored), the **scheduler** (picks a node for each pod) and the **controller manager** (makes StatefulSets create their pods). **CoreDNS** answers the RAC host names. No database runs here.
- **op-worker1 and op-worker2.** Each runs one RAC pod. A pod here behaves like a small RAC server: `systemd` runs inside it, and Grid Infrastructure, ASM, the listeners and one database instance all live in

the same container. The pods are **Guaranteed** (requests equal limits), so Kubernetes never squeezes their memory.

- **The operator.** One controller pod that watches the `RacDatabase` object. It runs on whichever worker Kubernetes chose; it doesn't need to be near the database.
- **Shared ASM disks.** The same three disks are attached to both worker VMs, so both pods see the same blocks. That's what makes it a RAC cluster rather than two separate databases.
- **Kubernetes objects.** You create four things (the `RacDatabase`, two Secrets, two network definitions, a node label); the operator creates the rest.

Why two private lines between the pods: RAC nodes constantly exchange heartbeats and data blocks (the **interconnect**). Two separate private networks give Grid two paths, so the cluster survives the loss of one.

See it on the lab

Commands from this lab's build and what they printed (... marks cut text). All are read-only except the `kubectl apply` in section 5, which creates the database.

The three nodes, seen from Windows through the API port forward

Look for: three nodes `Ready`, one `control-plane` and two `worker`, all v1.36.5.

COMMAND

```
# Git Bash on the Windows PC  (kop = kubectl with this lab's kubeconfig, API via 127.0.0.1:6444)
kop get nodes
```

THIS LAB'S OUTPUT

```
NAME              STATUS   ROLES           AGE   VERSION
op-controlplane   Ready    control-plane   12m   v1.36.5
op-worker1        Ready    worker          11m   v1.36.5
op-worker2        Ready    worker          11m   v1.36.5
```

The database object and the two RAC pods, one per worker

Look for: `racdb` `OPEN` and `AVAILABLE`; `racnode1-0` on op-worker1 and `racnode2-0` on op-worker2, both `1/1 Running`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac get racdatabase racdb; kubectl -n rac get pods -o wide
```

THIS LAB'S OUTPUT

```
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE
racnode1-0   1/1     Running   0          11h   10.244.1.6   op-worker1
racnode2-0   1/1     Running   0          11h   10.244.2.4   op-worker2
```

One database, one instance in each pod

Look for: `ORCLCDB1` on racnode1-0 and `ORCLCDB2` on racnode2-0, both open, the service on both.

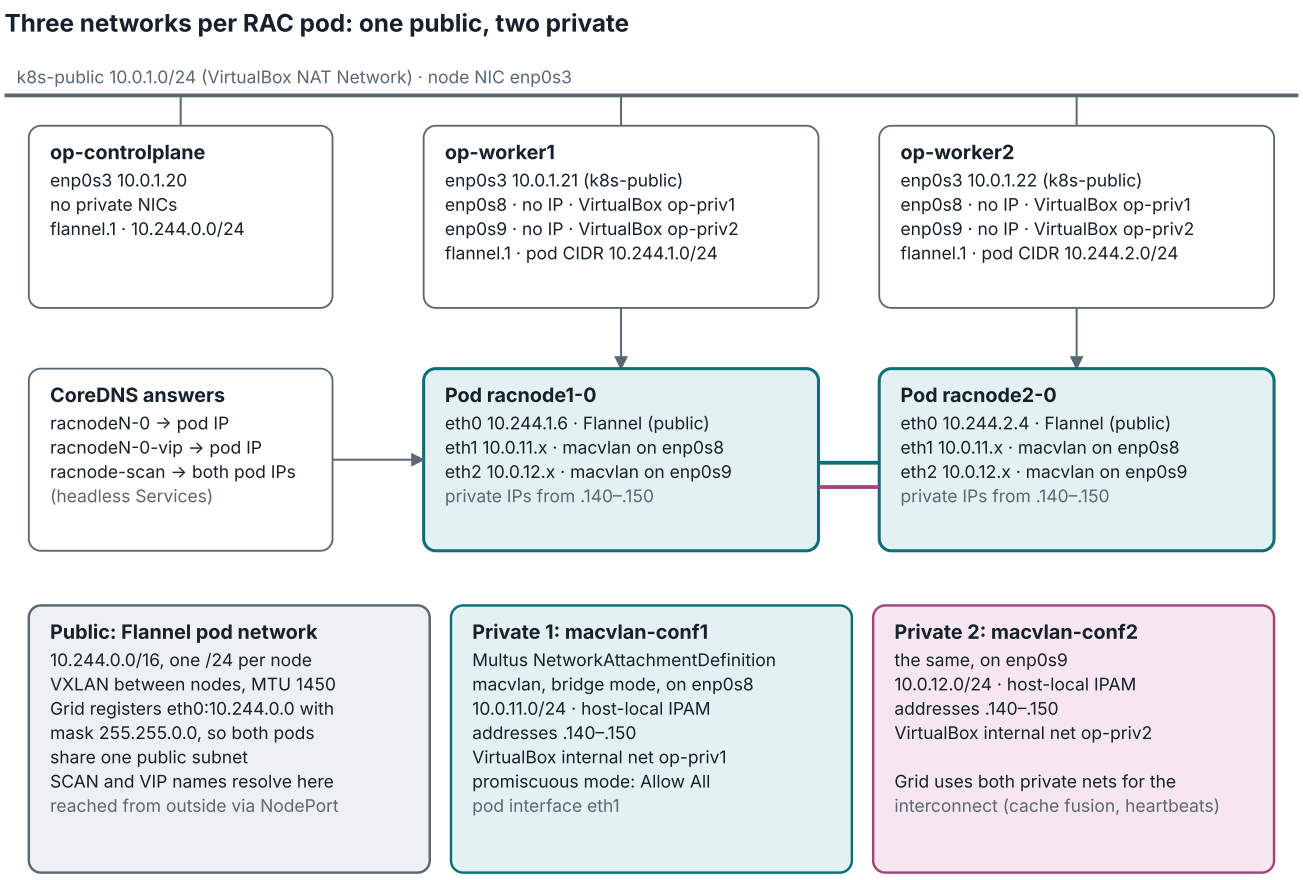
COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status database -d ORCLCDB -v; srvctl
status service -d ORCLCDB"
```

THIS LAB'S OUTPUT

```
Instance ORCLCDB1 is running on node racnode1-0 with online services orclpdbsvc. Instance status: Open.
Instance ORCLCDB2 is running on node racnode2-0 with online services orclpdbsvc. Instance status: Open.
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
```

3 Networks



Node NICs on top, pod interfaces below. The private interfaces get their own MAC and IP on top of a node NIC that has no IP.

Every RAC node needs one **public** network (clients, SCAN, VIPs) and at least one **private** network (the interconnect). In Kubernetes a pod normally has only one interface, so the design uses two network plugins:

- **Flannel** gives each pod its normal interface, `eth0`, on the pod network 10.244.0.0/16. Each node gets its own /24 (10.244.1.0/24 on op-worker1, 10.244.2.0/24 on op-worker2), and Flannel tunnels between nodes with VXLAN. The two pods are therefore on *different* /24s; Oracle's setup script registers the public network as `10.244.0.0` with mask `255.255.0.0` so Grid sees one public subnet.
- **Multus** lets a pod have extra interfaces, described by **NetworkAttachmentDefinitions** (`macvlan-conf1`, `macvlan-conf2`). Each one uses **macvlan** on a worker NIC that has no IP of its own (`enp0s8`, `enp0s9`): the pod's `eth1` and `eth2` get their own MAC and IP (10.0.11.x, 10.0.12.x) directly on the VirtualBox internal networks `op-priv1` and `op-priv2`. Promiscuous mode on those VirtualBox NICs must be **Allow All**, or the extra MAC addresses are dropped.

Names instead of addresses. The operator creates **headless Services** (Services without a virtual IP), so CoreDNS returns the pod's own IP for `racnode1-0`, `racnode2-0`, their `-vip` names, and both pod IPs for `racnode-scan`. Grid is installed against these names, so a pod that is recreated with a new IP keeps working.

VIPs are the pod IPs. On a classic RAC each node has a spare virtual IP that moves to the other node on failure. A Flannel pod has exactly one IP, so here `racnodeN-0-vip` resolves to the pod itself; when a

pod dies, its VIP dies with it. Clients still fail over through **SCAN** and the service running on the other instance.

From outside the cluster (Windows). A RAC client connects in two steps: first to a SCAN listener, which then **redirects** it to the node listener of the instance it picks, at the address that instance registered (`local_listener`). Out of the box both steps use addresses only the cluster can reach: SCAN on 1521 behind the operator's NodePort `racnode-scan-lsnr` (31521), and node listeners on the pod IPs, port 1522. So the redirect address was changed so that the pods and Windows can both reach it:

	ORCLCDB1 (RACNODE1-0)	ORCLCDB2 (RACNODE2-0)
extra node-listener port	31522	31523
`local_listener` (explicit, per instance)	`racnode1-0-vip.rac.svc.cluster.local:31522`	`racnode2-0-vip.rac.svc.cluster.local:31523`
that name inside the cluster	the pod IP (headless Service)	the pod IP
NodePort Service	`racnode1-0-lsnr-ext` 31522	`racnode2-0-lsnr-ext` 31523
on Windows	`hosts` → 127.0.0.1, forward 31522 → 10.0.1.21	`hosts` → 127.0.0.1, forward 31523 → 10.0.1.21

Plus a forward 127.0.0.1:31521 → 10.0.1.21 for SCAN. **Each node gets its own port** because the port SCAN hands out must be the same number on Windows, on the NodePort and on the listener, and Windows reaches both nodes through the one address 127.0.0.1. Once `local_listener` is set explicitly, Grid's agent stops rewriting it (it rewrote it when the listener's ports changed, and left it alone after). A client on Windows then connects to `127.0.0.1:31521/orclpdbsvc` and lands on either instance.

See it on the lab

Commands from this lab's build and what they printed (... marks cut text). All are read-only except the `kubectl apply` in section 5, which creates the database.

The SSH port forwards on the NAT network

Look for: `k8s-public` is 10.0.1.0/24, and ports 2220–2222 lead to the three VMs (.20, .21, .22).

COMMAND

```
# PowerShell on the Windows PC
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
& $VB natnetwork list
```

THIS LAB'S OUTPUT

```
Name:         k8s-public
Enabled:      Yes
Network:      10.0.1.0/24
Gateway:      10.0.1.1
DHCP Server:  No
…
Port-forwarding (ipv4)
        …                                   (rules for other VMs on this network left out)
        ssh-opcp:tcp:[]:2220:[10.0.1.20]:22
        ssh-opw1:tcp:[]:2221:[10.0.1.21]:22
        ssh-opw2:tcp:[]:2222:[10.0.1.22]:22
```

The worker NICs that carry the private networks

Look for: on each worker, NIC 2 on `op-priv1` and NIC 3 on `op-priv2`, both with `Promisc Policy: allow-all` (macvlan needs it).

COMMAND

```
# PowerShell on the Windows PC
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
foreach ($w in "op-worker1","op-worker2") { "--- $w"; & $VB showvminfo $w | Select-String "^NIC [23]:" }
```

THIS LAB'S OUTPUT

```
--- op-worker1
NIC 2:   MAC: 080027C33702, Attachment: Internal Network 'op-priv1', … Promisc Policy: allow-all, …
NIC 3:   MAC: 08002743BD30, Attachment: Internal Network 'op-priv2', … Promisc Policy: allow-all, …
--- op-worker2
NIC 2:   MAC: 0800279D8872, Attachment: Internal Network 'op-priv1', … Promisc Policy: allow-all, …
NIC 3:   MAC: 0800279E4885, Attachment: Internal Network 'op-priv2', … Promisc Policy: allow-all, …
```

The two Multus network definitions

Look for: `macvlan-conf1` (enp0s8, 10.0.11.0/24) and `macvlan-conf2` (enp0s9, 10.0.12.0/24) in namespace `rac`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac get network-attachment-definitions
```

THIS LAB'S OUTPUT

```
NAME           AGE
macvlan-conf1  0s
macvlan-conf2  0s
```

A test pod with all three interfaces, and the private networks working between workers

Look for: `eth0` on Flannel (10.244.1.x, MTU 1450 because of VXLAN), `eth1` 10.0.11.x and `eth2` 10.0.12.x from macvlan; 0% loss across both private networks. These were two short-lived busybox test pods (`nettest-w1` on op-worker1, `nettest-w2` on op-worker2), deleted afterwards.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec nettest-w1 -- ip -4 addr
kubectl -n rac exec nettest-w1 -- ping -c3 10.0.11.149
kubectl -n rac exec nettest-w1 -- ping -c3 10.0.12.149
```

THIS LAB'S OUTPUT

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    inet 127.0.0.1/8 scope host lo
2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue qlen 1000
    inet 10.244.1.2/24 brd 10.244.1.255 scope global eth0
3: eth1@eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue qlen 1000
    inet 10.0.11.148/24 brd 10.0.11.255 scope global eth1
4: eth2@eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue qlen 1000
    inet 10.0.12.148/24 brd 10.0.12.255 scope global eth2
PING 10.0.11.149 (10.0.11.149): 56 data bytes
64 bytes from 10.0.11.149: seq=0 ttl=64 time=0.855 ms
…
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.0.12.149 (10.0.12.149): 56 data bytes
…
3 packets transmitted, 3 packets received, 0% packet loss
```

The names the operator created

Look for: headless Services (`CLUSTER-IP None`) for each node, each VIP and the SCAN; one NodePort Service, `racnode-scan-lsnr`, 1521 → 31521.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac get racdatabase,sts,svc,pvc
```

THIS LAB'S OUTPUT

```
…
NAME                         TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
service/racnode-scan         ClusterIP   None          <none>        <none>           30s
service/racnode-scan-lsnr    NodePort    10.99.59.33   <none>        1521:31521/TCP   30s
service/racnode1-0           ClusterIP   None          <none>        <none>           30s
service/racnode1-0-vip       ClusterIP   None          <none>        <none>           30s
service/racnode2-0           ClusterIP   None          <none>        <none>           30s
service/racnode2-0-vip       ClusterIP   None          <none>        <none>           30s
…
```

What Grid found inside the pod (key lines of the setup log on racnode1-0)

Look for: the setup script sees `eth0` with the pod IP (public) and `eth1` with a macvlan IP (private).

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

THIS LAB'S OUTPUT

```
01:23:20  oracommon - getnwlist :The network card: eth0 for the ip: 10.244.1.6
01:23:20  oracommon - getnwlist :The network card: eth1 for the ip: 10.0.11.140
```

Where the VIPs and SCAN listeners run

Look for: each VIP is its own pod's IP; one SCAN listener per node. (The first lines of this check, CRS online and both nodes `Active`, are in section 4's root.sh proof and section 6.)

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "crsctl check cluster -all; olsnodes -n -s; crsctl
query css votedisk; asmcmd lsdg; srvctl status nodeapps; srvctl status scan_listener" 2>/dev/null
```

THIS LAB'S OUTPUT

```
…
VIP 10.244.1.6 is enabled
VIP 10.244.1.6 is running on node: racnode1-0
VIP 10.244.2.4 is enabled
VIP 10.244.2.4 is running on node: racnode2-0
…
SCAN listener LISTENER_SCAN1 is running on node racnode1-0
SCAN listener LISTENER_SCAN2 is running on node racnode2-0
```

From Windows through SCAN to both instances

Look for: ten connections to `127.0.0.1:31521/orclpdbsvc` from Python (thin mode), split over `ORCLCDB1` and `ORCLCDB2`; the SYSTEM password is asked at a hidden prompt.

COMMAND

```
# PowerShell on the Windows PC  (scan_test.py: ten connections, prints instance, host and container)
python D:\rac-k8s-poc-operator\scan_test.py
```

THIS LAB'S OUTPUT

```
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB1  racnode1-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB1  racnode1-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB2  racnode2-0  ORCLPDB
 7  ORCLCDB1  racnode1-0  ORCLPDB
 8  ORCLCDB1  racnode1-0  ORCLPDB
 9  ORCLCDB2  racnode2-0  ORCLPDB
10  ORCLCDB1  racnode1-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 5, 'ORCLCDB1': 5}
```

What SCAN hands out now

Look for: both SCAN listeners list each instance at its `-vip` name and its own port, the address Windows can reach.

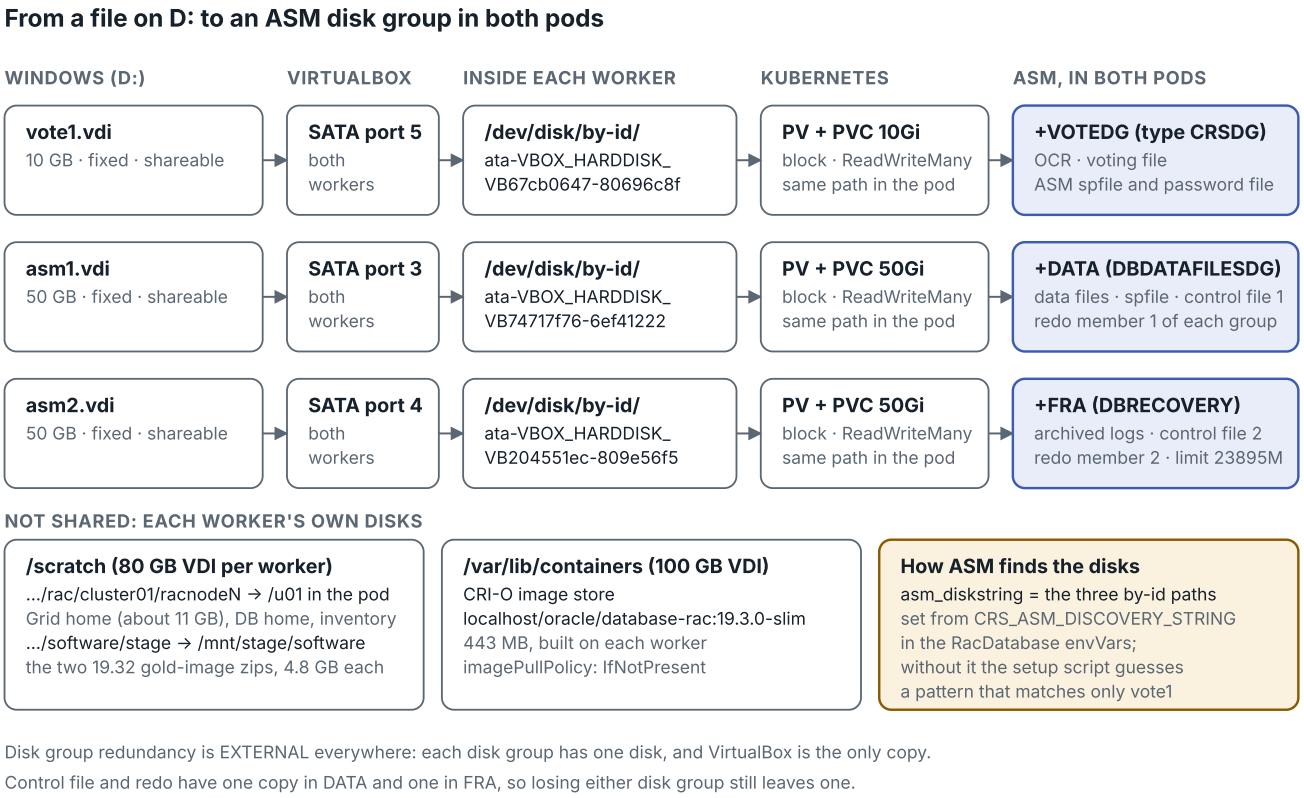
COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "lsnrctl services LISTENER_SCAN1" | awk '/Service
"orclpdbsvc"/{f=1;print;next} /^Service "/{f=0} f && /Instance|ADDRESS/'
```

THIS LAB'S OUTPUT

```
Service "orclpdbsvc" has 2 instance(s).
  Instance "ORCLCDB1", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))
```

4 Storage



Each row is one shared disk. The same by-id name appears on both workers, so one PV serves both pods.

Shared disks. Each shared disk is a **fixed-size, shareable** VDI file attached to both worker VMs on the same SATA port. VirtualBox gives the disk a serial number derived from the VDI's UUID, so Linux shows it under the same `/dev/disk/by-id/ata-VBOX_HARDDISK_VB…` name on both workers. Those names, not `/dev/sdX`, go into the `RacDatabase`, because `sdX` letters depend on discovery order.

From disk to ASM. The operator first runs a short-lived disk-check pod on each worker to read every disk's size, then creates one **PersistentVolume** (block mode, ReadWriteMany) and one **PersistentVolumeClaim** per disk, and hands the device to both pods at the same path. Inside the pods, ASM builds one **disk group** per disk:

DISK GROUP	TYPE IN THE YAML	HOLDS
`+VOTEDG` (10 GB)	`CRSDG`	the cluster registry (OCR), the voting file, ASM's spfile and password file; created by Grid's `root.sh`
`+DATA` (50 GB)	`DBDATAFILESDG`	data files, the database spfile, control file copy 1, one member of each redo group
`+FRA` (50 GB)	`DBRECOVERY`	the fast recovery area: archived logs, control file copy 2, the second member of each redo group; limit 23895M

Why three disk groups: the cluster's own files stay small and separate from the database, and a full recovery area can't take the cluster down with it. Each disk group has one disk and **EXTERNAL** redundancy, because in this lab there is nothing to mirror to: every disk is a file on the same D: drive.

Not shared: each worker's `/scratch` disk holds that node's Oracle homes (mounted as `/u01` in the pod) and the staged gold-image zips; `/var/lib/containers` holds the container image. The homes are per node: each pod unzips and installs its own copy.

See it on the lab

Commands from this lab's build and what they printed (... marks cut text). All are read-only except the `kubectl apply` in section 5, which creates the database.

The three shared VDI files, each attached to both workers

Look for: `Type: shareable`, the right capacity, and both `op-worker1` and `op-worker2` under "In use by VMs".

COMMAND

```
# PowerShell on the Windows PC
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
foreach ($d in "vote1","asm1","asm2") { & $VB showmediuminfo disk "$Base\shared\$d.vdi" | Select-String
'^(UUID|Location|Type|Capacity|In use by VMs):|op-worker' }
```

THIS LAB'S OUTPUT

```
UUID:           67cb0647-d253-4702-8823-e8628f6c6980
Type:           shareable
Capacity:       10240 MBytes
In use by VMs:  op-worker1 (UUID: 8164c82b-4b03-49a4-b193-671b310c6962)
                op-worker2 (UUID: 05c9cfea-2eba-4f10-8c54-9c1b2919b592)
UUID:           74717f76-da51-4430-87ce-3b192212f46e
Type:           shareable
Location:       D:\VMs\rac-op\shared\asm1.vdi
In use by VMs:  op-worker1 (UUID: 8164c82b-…)
                op-worker2 (UUID: 05c9cfea-…)
UUID:           204551ec-9ae3-4871-b2b9-8c68f5569e80
Type:           shareable
Location:       D:\VMs\rac-op\shared\asm2.vdi
In use by VMs:  op-worker1 (UUID: 8164c82b-…)
                op-worker2 (UUID: 05c9cfea-…)
```

The by-id names, derived from the VDI UUIDs and the same on both workers

Look for: `VB` + the first 8 characters of each UUID above; these three paths are what the `RacDatabase` lists.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
cat /root/rac-op/asm-disks
```

THIS LAB'S OUTPUT

```
VOTE=/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f
DATA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222
FRA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
```

One PV and PVC per disk, created by the operator

Look for: three claims `Bound`, 10Gi and 2 × 50Gi, all `RWX` (ReadWriteMany), so both pods can use them.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac get racdatabase,sts,svc,pvc
```

THIS LAB'S OUTPUT

```
…
NAME                                          STATUS   VOLUME                 CAPACITY   ACCESS MODES
persistentvolumeclaim/asm-pvc-833f4ea4-racdb   Bound    asm-pv-833f4ea4-racdb   10Gi       RWX
persistentvolumeclaim/asm-pvc-c51c7cc4-racdb   Bound    asm-pv-c51c7cc4-racdb   50Gi       RWX
persistentvolumeclaim/asm-pvc-eaac2e15-racdb   Bound    asm-pv-eaac2e15-racdb   50Gi       RWX
```

ASM's discovery string, covering all three disks

Look for: `parameter:` (the ASM spfile) and `profile:` (the cluster profile) both list the three by-id paths.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "asmcmd dsget"
```

THIS LAB'S OUTPUT

```
parameter:/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f, /dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-
6ef41222, /dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
profile:/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f,/dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-
6ef41222,/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
```

The three disk groups

Look for: all `MOUNTED EXTERN`; `DATA` and `FRA` 51200 MB, `VOTEDG` 10240 MB and the only one with `Voting_files Y`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "asmcmd lsdg"
```

THIS LAB'S OUTPUT

```
State    Type    Rebal  Sector  Logical_Sector  Block       AU  Total_MB  Free_MB  Req_mir_free_MB
Usable_file_MB  Offline_disks  Voting_files  Name
MOUNTED  EXTERN  N         512             512   4096  1048576     51200    44128                0
44128              0             N  DATA/
MOUNTED  EXTERN  N         512             512   4096  1048576     51200    46197                0
46197              0             N  FRA/
MOUNTED  EXTERN  N         512             512   4096  4194304     10240     9884                0
9884               0             Y  VOTEDG/
```

The voting file on VOTEDG (from Grid's root.sh log on racnode1-0)

Look for: one voting disk, `ONLINE`, on the vote1 path, in `[VOTEDG]`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'f=$(ls -t /u01/app/19c/grid/install/root_racnode1-
0_*.log 2>/dev/null | head -1); [ -n "$f" ] && cat "$f"'
```

THIS LAB'S OUTPUT

```
…
Successful addition of voting disk f2a52506ee824fecbf0b6936be04ac26.
Successfully replaced voting disk group with +VOTEDG.
CRS-4256: Updating the profile
CRS-4266: Voting file(s) successfully replaced
##  STATE    File Universal Id                File Name Disk group
--  -----    -----------------                --------- ---------
 1. ONLINE   f2a52506ee824fecbf0b6936be04ac26 (/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f) [VOTEDG]
Located 1 voting disk(s).
…
```

Control files and redo, one copy in each of DATA and FRA

Look for: two control files (`+DATA` and `+FRA`), and two members per redo group, one in each disk group.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c 'export ORACLE_SID=ORCLCDB1; sqlplus -s / as
sysdba <<EOF
set lines 200 pages 100
col member for a70
col name for a60
select name from v\$controlfile;
select group#, status, member from v\$logfile order by 1, member;
EOF'
```

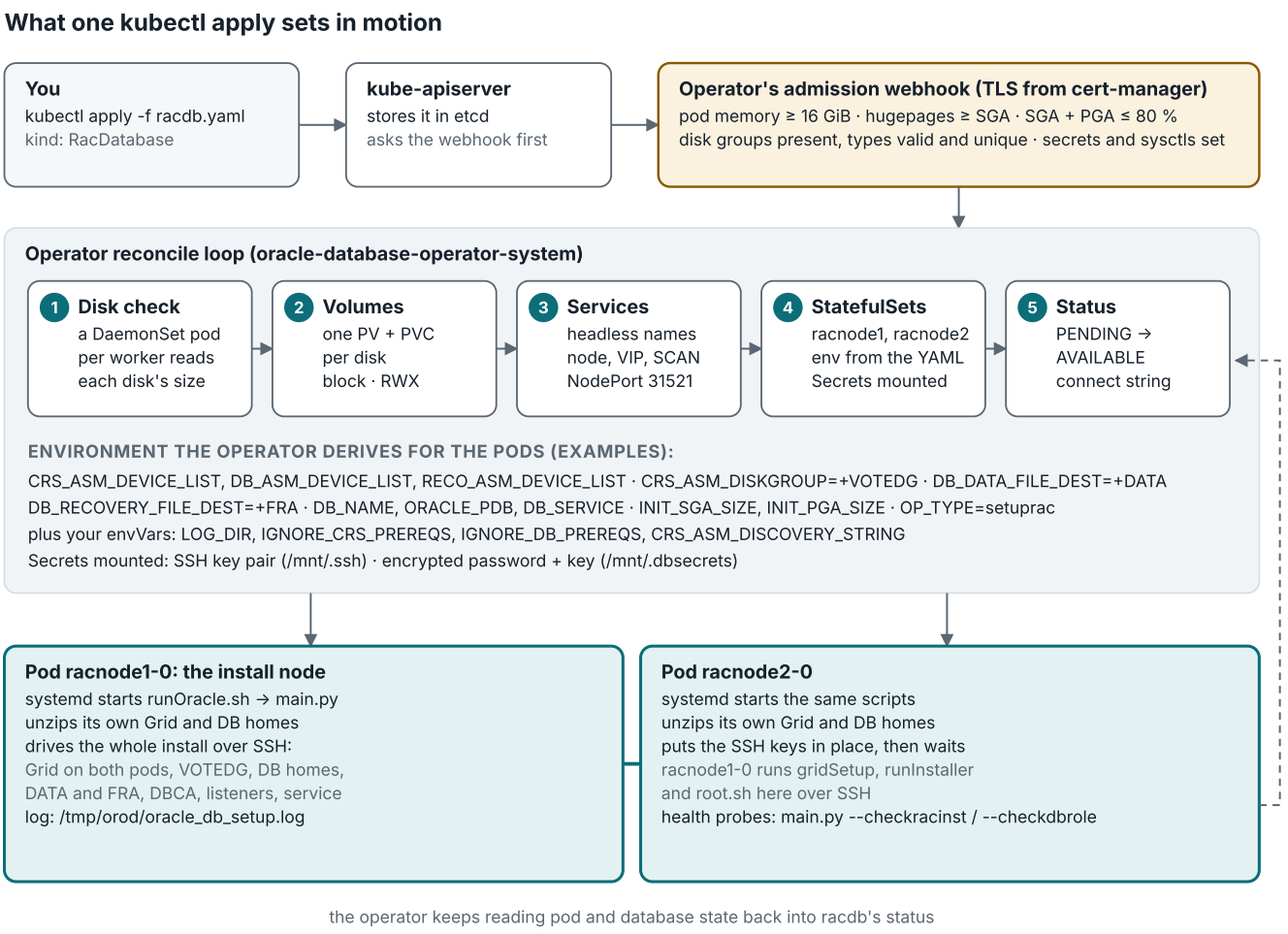
THIS LAB'S OUTPUT

```
NAME
------------------------------------------------------------
+DATA/ORCLCDB/CONTROLFILE/current.278.1247831575
+FRA/ORCLCDB/CONTROLFILE/current.267.1247831577

    GROUP# STATUS  MEMBER
---------- ------- --------------------------------------------------------------------------------
         1         +DATA/ORCLCDB/ONLINELOG/group_1.274.1247831211
         1         +FRA/ORCLCDB/ONLINELOG/group_1.258.1247796499
         2         +DATA/ORCLCDB/ONLINELOG/group_2.275.1247831285
         2         +FRA/ORCLCDB/ONLINELOG/group_2.257.1247796499
         3         +DATA/ORCLCDB/ONLINELOG/group_3.276.1247831353
         3         +FRA/ORCLCDB/ONLINELOG/group_3.259.1247800443
         4         +DATA/ORCLCDB/ONLINELOG/group_4.277.1247831421
         4         +FRA/ORCLCDB/ONLINELOG/group_4.260.1247800527

8 rows selected.
```

5 How the operator builds it



The operator turns the RacDatabase into ordinary Kubernetes objects; the database install itself runs inside the pods.

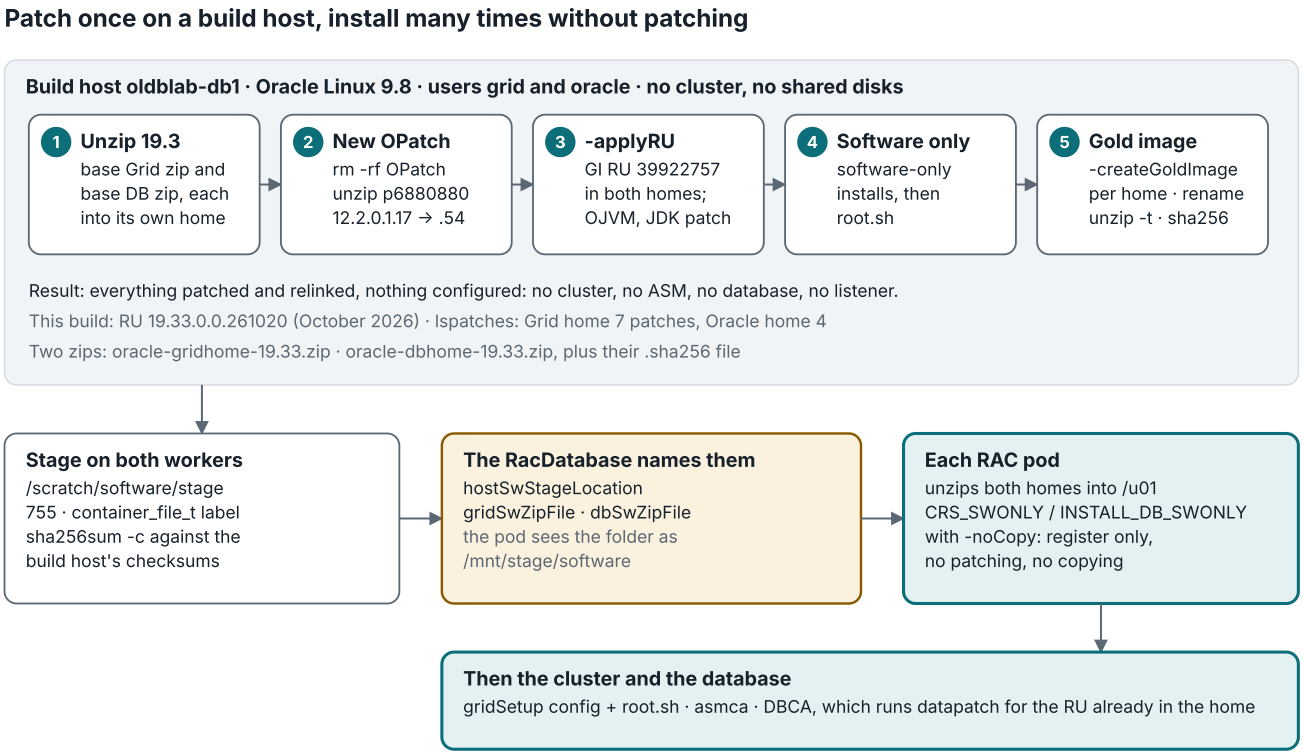
An operator is a program that runs inside Kubernetes and adds a new kind of object, here `RacDatabase`, together with the logic to make it real. Kubernetes stores your `RacDatabase` like any other object; the operator notices it and keeps creating or fixing ordinary objects until the cluster matches what you asked for. This loop is called **reconciling**.

- **Admission webhook.** Before the API server stores the object, it asks the operator's webhook. The webhook refuses settings that can't work, such as less than 16 GiB of pod memory, too few hugepages for the SGA, or missing disk groups. Kubernetes only calls webhooks over TLS; **cert-manager** issues that certificate.
- **Reconcile steps.** Disk check, volumes, Services and StatefulSets, in that order. Each RAC node gets its own **StatefulSet** with one pod, so `racnode1-0` and `racnode2-0` keep stable names and their own volumes across restarts.
- **Configuration as environment.** The operator translates the YAML into environment variables for the pods: the device lists per disk group, the disk group names, database name, SGA and PGA, plus your own `envVars`. The SSH keys and the encrypted password arrive as mounted Secrets; the password is decrypted only inside the pod, at install time.
- **The install itself runs in the pods.** The pod starts `systemd`, which runs Oracle's setup scripts (`runOracle.sh` → `main.py`). Both pods prepare themselves; `racnode1-0` is the install node and

drives every installer step on both pods over SSH. The operator only watches and reports.

- **Status.** The operator reads the pods and the database back into the `RacDatabase` status: `PENDING` during the install, then `OPEN … AVAILABLE` with the connect string `racnode-scan:1521/orclpdbsvc`.

The software: gold-image zips, patched once



Top: done once per Release Update on a plain Oracle Linux 9.8 server. Bottom: done by every RAC build; the pods only unzip and register the already patched homes.

Why gold images: a RAC pod only unzips the Grid and DB homes and registers them with software-only installs (`-noCopy`). If the zips were plain 19.3, every deployment would also have to replace OPatch and apply the Release Update on both nodes, adding a long patching step to an install that already takes hours. Instead the homes are patched **once per RU** on a separate Oracle Linux 9.8 server, installed software-only there, and packed with `-createGoldImage`. Every RAC build at that patch level reuses the two zips.

What goes into each home (the build on `oldblab-db1`, 28 October 2026, RU **19.33.0.0.261020**):

HOME	PATCHES AFTER THE BUILD (`OPATCH LSPATCHES`)
Grid	GI RU `39922757`, which brings Database RU `39915792`, OCW `39922783`, ACFS `39922802`, Tomcat `39107855`, DBWLM `39107825`; plus OJVM RU `39803650` and JDK `39902051`
DB	GI RU `39922757`, which brings Database RU `39915792` and OCW `39922783`; plus OJVM RU `39803650` and JDK `39902051`

Why the GI RU in the DB home too: it contains the Database RU and the OCW RU that a RAC database home needs, so both homes end up at exactly the same level. The OJVM RU is only needed in the DB home; this build also put it in the Grid home, which did no harm. Next time it goes into the DB home only, as a one-off after the RU.

Why it is quick to deploy: `-applyRU` patches the unzipped home before the installer runs, so the software-only install already uses the patched 19.33 installer, which knows Oracle Linux 9. The gold image is the home after that install: patched and relinked, not configured. In the pod, the operator's setup script unzips it and registers it again with `-noCopy`, and DBCA brings the new database's dictionary to the same patch level.

THE BUILD, IN SHORT (FULL STEPS AND OUTPUT: LAB GUIDE 8.0)

```
# oldblab-db1 as grid, in /u01/app/19.0.0/grid (the DB home is the same with runInstaller, as oracle)
unzip -q /u01/app/oracle/software/LINUX.X64_193000_grid_home.zip
rm -rf OPatch && unzip -q /u01/app/oracle/software/p6880880_190000_Linux-x86-64.zip
./gridSetup.sh -applyRU /u01/app/oracle/software/40121665/39922757/ -silent
./gridSetup.sh -applyOneOffs /u01/app/oracle/software/39902051 -silent
./gridSetup.sh -silent -waitforcompletion oracle.install.option=CRS_SWONLY ORACLE_BASE=/u01/app/grid \
  INVENTORY_LOCATION=/u01/app/oraInventory UNIX_GROUP_NAME=oinstall \
  oracle.install.asm.OSDBA=asmdba oracle.install.asm.OSOPER=asmoper oracle.install.asm.OSASM=asmadmin
#   (as root: /u01/app/oraInventory/orainstRoot.sh; /u01/app/19.0.0/grid/root.sh)
./gridSetup.sh -createGoldImage -destinationLocation /tmp/grid-gold -silent
```

THIS LAB'S OUTPUT (THE LAST LINE OF EACH -CREATEGOLDIMAGE)

```
Gold Image location: /tmp/grid-gold/grid_home_2026-10-28_03-48-55PM.zip
Gold Image location: /tmp/db-gold/db_home_2026-10-28_04-05-17PM.zip
```

The image the pods run: the RAC slim image

What it is: Oracle Linux 9 with the OS packages, users and groups Oracle needs, plus Oracle's own setup scripts (`runOracle.sh`, `main.py` and the modules they call), but **no** Oracle software. That is what "slim" means. The operator starts this image and the scripts in it install Grid and the database at run time, from the gold-image zips staged on the worker, into `/u01` on the worker's disk. The image stays small (443 MB) and the 11 GB homes live outside it.

Why on each worker, with no registry: each worker's CRI-O can only start images it has locally, and either RAC pod may land on either labelled worker. Podman builds straight into the image store CRI-O reads (`/var/lib/containers`), so the image never has to leave the worker it was built on.

Why each part of the build:

- `podman-docker`**:** Oracle's build script calls `docker`; this package makes `docker` run Podman. `/etc/containers/nodocker` silences its "emulating docker" notice.
- **A sparse clone:** only the RAC folder of Oracle's large `docker-images` repository is downloaded.
- `-v 19.3.0`**:** the 19c folder, used for every 19c Release Update, 19.32 included. Its `Containerfile` is based on Oracle Linux 9.
- `-i`**:** skip the checksum of install zips, since the slim image doesn't contain any.
- `--build-arg SLIMMING=true`**:** build without the Oracle software.
- `TMPDIR` **on** `/scratch`**:** Podman's temporary files go to the 80 GB disk instead of the small root disk.
- `crictl images`**:** if CRI-O lists the image, Kubernetes can start it.

The build downloads Oracle Linux packages; here it took under 3 minutes per worker (allow 10–20 on a slower connection). The two workers can build at the same time, in two windows.

COMMAND

```
# op-worker1, then op-worker2  (10-20 minutes each; connect: ssh -p 2221 root@127.0.0.1; for op-worker2: ssh -p 2222
root@127.0.0.1)
dnf install -y -q git podman-docker && touch /etc/containers/nodocker
mkdir -p /scratch/image/tmp && rm -rf /scratch/image/docker-images
git clone -q --depth 1 --filter=blob:none --sparse https://github.com/oracle/docker-images.git /scratch/image/docker-
images
cd /scratch/image/docker-images && git sparse-checkout set OracleDatabase/RAC/OracleRealApplicationClusters && git log
-1 --format='docker-images commit %h, %cd'
cd OracleDatabase/RAC/OracleRealApplicationClusters/containerfiles
export TMPDIR=/scratch/image/tmp
./buildContainerImage.sh -v 19.3.0 -t localhost/oracle/database-rac:19.3.0-slim -i -o '--build-arg SLIMMING=true' >
/scratch/image/build.log 2>&1; echo "build exit $?"
tail -8 /scratch/image/build.log
podman images localhost/oracle/database-rac
crictl images | grep database-rac
```

THIS LAB'S OUTPUT (THE IMAGE IDS DIFFER BECAUSE EACH WORKER BUILDS ITS OWN COPY)

```
[root@op-worker1 containerfiles]#
build exit 0
e9302892b20d51de90d6eaba11a31d25a830538ca1c7fc432cc65620ec895556

  Oracle Database container Image for Real Application Clusters (RAC) version 19.3.0 is ready to be extended:

    --> localhost/oracle/database-rac:19.3.0-slim

  Build completed in 168 seconds.

REPOSITORY                    TAG          IMAGE ID      CREATED       SIZE
localhost/oracle/database-rac  19.3.0-slim  e9302892b20d  1 second ago  443 MB
localhost/oracle/database-rac                19.3.0-slim         e9302892b20d5       443MB

[root@op-worker2 containerfiles]#
build exit 0
ad073d6cdfc02e06a761d0ede9daa43c9972042bdc1e5786cc19752e82ad5b68
  …
    --> localhost/oracle/database-rac:19.3.0-slim

  Build completed in 169 seconds.

REPOSITORY                    TAG          IMAGE ID      CREATED       SIZE
localhost/oracle/database-rac  19.3.0-slim  ad073d6cdfc0  1 second ago  443 MB
localhost/oracle/database-rac                19.3.0-slim         ad073d6cdfc02       443MB
```

The software beside it: the two gold-image zips are copied to the workers. The database running in this lab was built from the 19.32 zips (`oracle-gridhome-19.32.zip`, `oracle-dbhome-19.32.zip`, 4.8 GB each; it reports `19.32.0.0.0`); a build from the new 19.33 zips uses `oracle-gridhome-19.33.zip` and `oracle-dbhome-19.33.zip`. They go to `/scratch/software/stage` on both workers and checked with `sha256sum`. The pod sees that folder as `/mnt/stage/software` and unzips each home from there.

The RacDatabase this lab applied

Everything the operator builds comes from this one object. The block reads the three disk paths saved on op-controlplane, writes `/root/rac-op/racdb.yaml` with them filled in, and asks the API server to check it without creating anything (`--dry-run=server`, which runs the operator's webhook). `kubectl apply -f /root/rac-op/racdb.yaml` then creates it for real.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
source /root/rac-op/asm-disks; echo "VOTE=$VOTE"; echo "DATA=$DATA"; echo "FRA=$FRA"
cat > /root/rac-op/racdb.yaml <<EOF
apiVersion: database.oracle.com/v4
kind: RacDatabase
metadata:
  name: racdb
  namespace: rac
spec:
  instanceDetails:
    nodeCount: 2
    racHostSwLocation: /scratch/rac/cluster01
    racNodeName: racnode
    privateIPDetails:
      - { name: macvlan-conf1, interface: eth1 }
      - { name: macvlan-conf2, interface: eth2 }
    workerNodeSelector: { raccluster: raccluster01 }
  envVars:
    - { name: LOG_DIR, value: /tmp/orod }
    - { name: IGNORE_CRS_PREREQS, value: "true" }
    - { name: IGNORE_DB_PREREQS, value: "true" }
    - { name: CRS_ASM_DISCOVERY_STRING, value: "$VOTE,$DATA,$FRA" }
  asmDiskGroupDetails:
    - { name: VOTEDG, redundancy: EXTERNAL, type: CRSDG, disks: [ $VOTE ] }
    - { name: DATA, redundancy: EXTERNAL, type: DBDATAFILESDG, disks: [ $DATA ] }
    - { name: FRA, redundancy: EXTERNAL, type: DBRECOVERY, disks: [ $FRA ] }
  sshKeySecret: { name: ssh-key-secret, privKeySecretName: ssh-privkey, pubKeySecretName: ssh-pubkey }
  dbSecret:
    name: db-user-pass
    keyFileName: key.pem
    pwdFileName: pwdfile.enc
    encryptionType: pkeyutl
    pkeyopt: "rsa_padding_mode:oaep;rsa_oaep_md:sha256;rsa_mgf1_md:sha256"
  image: localhost/oracle/database-rac:19.3.0-slim
  imagePullPolicy: IfNotPresent
  scanSvcName: racnode-scan
  scanSvcTargetPort: 31521
  serviceDetails: { name: orclpdbsvc }
  resources:
    requests: { cpu: "3", memory: 16Gi, hugepages-2Mi: 4Gi }
    limits:   { cpu: "3", memory: 16Gi, hugepages-2Mi: 4Gi }
  securityContext:
    sysctls:
      - { name: kernel.shmall, value: "2097152" }
      - { name: kernel.sem, value: "250 32000 100 128" }
      - { name: kernel.shmmax, value: "8589934592" }
      - { name: kernel.shmmni, value: "4096" }
      - { name: net.ipv4.conf.all.rp_filter, value: "2" }
  configParams:
    gridHome: /u01/app/19c/grid
    gridBase: /u01/app/grid
    dbHome: /u01/app/oracle/product/19c/dbhome_1
    dbBase: /u01/app/oracle
    inventory: /u01/app/oraInventory
    hostSwStageLocation: /scratch/software/stage
    gridSwZipFile: oracle-gridhome-19.32.zip
    dbSwZipFile: oracle-dbhome-19.32.zip
    dbName: ORCLCDB
    pdbName: ORCLPDB
    sgaSize: "3G"
    pgaSize: "1G"
    cpuCount: 3
EOF
kubectl apply --dry-run=server -f /root/rac-op/racdb.yaml
```

THIS LAB'S OUTPUT

```
VOTE=/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f
DATA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222
```

```
FRA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
racdatabase.database.oracle.com/racdb created (server dry run)
```

One difference from the run: the first apply in this lab didn't have the `CRS_ASM_DISCOVERY_STRING` line. That is what caused the ORA-15014 stop (the red bar in section 6). The YAML shown here is the corrected one.

What each part becomes:

YAML	WHAT THE OPERATOR OR THE SETUP SCRIPT MAKES OF IT
`instanceDetails.nodeCount: 2`, `racNodeName: racnode`	two StatefulSets, `racnode1` and `racnode2`, one pod each (`racnode1-0`, `racnode2-0`), plus headless Services `racnodeN-0` and `racnodeN-0-vip`
`racHostSwLocation: /scratch/rac/cluster01`	a host directory per node, `…/racnode1` and `…/racnode2`, mounted as `/u01` in the pod for the Oracle homes and inventory
`privateIPDetails`	the Multus networks `macvlan-conf1` and `macvlan-conf2` attached to each pod as `eth1` and `eth2`, the interconnect
`workerNodeSelector: { raccluster: raccluster01 }`	the pods only go to workers with that label (both workers have it)
`envVars`	passed unchanged into the pods: the setup log directory, skipping the optional prerequisite checks that always fail in a container, and ASM's discovery string covering all three disks
`asmDiskGroupDetails`	a disk-check pod per worker, one PV and PVC per disk, the device in both pods at the same path, and the device lists and disk group names in the pod environment; Grid's `root.sh` creates `VOTEDG`, `asmca` creates `FRA` and `DATA`
`sshKeySecret`, `dbSecret`	the Secrets mounted at `/mnt/.ssh` and `/mnt/.dbsecrets`; the password is decrypted inside the pod with the same OpenSSL settings used to encrypt it
`image`, `imagePullPolicy: IfNotPresent`	the RAC image built on each worker; `IfNotPresent` because it exists only locally
`scanSvcName`, `scanSvcTargetPort: 31521`	the headless `racnode-scan` Service and the NodePort Service `racnode-scan-lsnr` (1521 → 31521)
`serviceDetails: { name: orclpdbsvc }`	the database service on both instances, part of the connect string `racnode-scan:1521/orclpdbsvc`

YAML	WHAT THE OPERATOR OR THE SETUP SCRIPT MAKES OF IT
`resources`	Guaranteed pods (requests equal limits): 3 CPU, 16 GiB memory, 4 GiB of 2 MiB hugepages; the webhook checks these against the SGA and PGA
`securityContext.sysctls`	kernel settings inside the pod; `kernel.shmmax` (8 GiB) must be at least the SGA, `kernel.shmall` at least SGA / 4096 pages
`configParams`	the Oracle homes, the staged 19.32 gold-image zips, the database `ORCLCDB` with PDB `ORCLPDB`, SGA 3 GB, PGA 1 GB, `cpu_count` 3

See it on the lab

Commands from this lab's build and what they printed (... marks cut text). All are read-only except the `kubectl apply` in section 5, which creates the database.

cert-manager, which issues the webhook's TLS certificate

Look for: three cert-manager pods `1/1 Running`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n cert-manager get pods
```

THIS LAB'S OUTPUT

```
NAME                                       READY   STATUS    RESTARTS   AGE
cert-manager-69c7fcbf78-vv5dd              1/1     Running   0          48s
cert-manager-cainjector-69f8c8cdbf-ptnn2   1/1     Running   0          48s
cert-manager-webhook-84fd89df64-bkjmm      1/1     Running   0          48s
```

The new object type, and the one operator pod

Look for: the `racdatabases.database.oracle.com` CRD exists; one controller-manager pod `Running`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl get crd racdatabases.database.oracle.com
kubectl -n oracle-database-operator-system get pods
```

THIS LAB'S OUTPUT

```
NAME                              CREATED AT
racdatabases.database.oracle.com   2026-10-27T12:06:53Z
NAME                                                        READY   STATUS    RESTARTS   AGE
oracle-database-operator-controller-manager-54cf89dd45-f644n   1/1     Running   1          12h
```

The operator's permissions outside its namespaces

Look for: every line `yes`: it may read nodes, create and delete PVs, read the Multus networks, and create StatefulSets and RacDatabases in `rac`.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
SA=system:serviceaccount:oracle-database-operator-system:oracle-database-operator-controller-manager
for c in "list nodes" "create persistentvolumes" "delete persistentvolumes" "list network-attachment-
definitions.k8s.cni.cncf.io -n rac" "create statefulsets.apps -n rac" "create racdatabases.database.oracle.com
-n rac"; do
  printf '%-62s %s\n' "$c" "$(kubectl auth can-i $c --as=$SA)"
done
```

THIS LAB'S OUTPUT

```
Warning: resource 'nodes' is not namespace scoped
list nodes                                                  yes
Warning: resource 'persistentvolumes' is not namespace scoped
create persistentvolumes                                    yes
Warning: resource 'persistentvolumes' is not namespace scoped
delete persistentvolumes                                    yes
list network-attachment-definitions.k8s.cni.cncf.io -n rac  yes
create statefulsets.apps -n rac                             yes
create racdatabases.database.oracle.com -n rac              yes
```

The webhook checks the YAML without creating anything

Look for: `created (server dry run)`: the API server and the operator's webhook accepted it; nothing was stored.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl apply --dry-run=server -f /root/rac-op/racdb.yaml
```

THIS LAB'S OUTPUT

```
racdatabase.database.oracle.com/racdb created (server dry run)
```

Thirty seconds after the real apply: the reconcile steps' results

Look for: `racdb` `PENDING`; two StatefulSets; the disk-check pods finishing (their `Error` while terminating is normal); the RAC pods `Running` but `0/1` ready until the install is done.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl apply -f /root/rac-op/racdb.yaml
sleep 30; kubectl -n rac get racdatabase,sts,svc,pvc
kubectl -n rac get pods -o wide -w
```

THIS LAB'S OUTPUT

```
racdatabase.database.oracle.com/racdb created
NAME                                  DBNAME    DBSTATE   ROLE          VERSION       PDB CONNECT STR   STATE
racdatabase.database.oracle.com/racdb   ORCLCDB   PENDING   NOT_DEFINED   NOT_DEFINED   NOT_DEFINED
PENDING

NAME                         READY   AGE
statefulset.apps/racnode1   0/1     14s
statefulset.apps/racnode2   0/1     14s
…                                        (Services and PVCs: see sections 3 and 4)
NAME                          READY   STATUS        RESTARTS   AGE   IP           NODE
disk-check-daemonset-9p4gz   1/1     Terminating   0          30s   10.244.1.5   op-worker1
disk-check-daemonset-s7fhr   1/1     Terminating   0          30s   10.244.2.3   op-worker2
racnode1-0                   0/1     Running       0          14s   10.244.1.6   op-worker1
racnode2-0                   0/1     Running       0          14s   10.244.2.4   op-worker2
disk-check-daemonset-9p4gz   0/1     Error         0          42s   10.244.1.5   op-worker1
disk-check-daemonset-s7fhr   0/1     Error         0          42s   10.244.2.3   op-worker2
```

Configuration arriving in the pod (key lines of the setup log on racnode1-0)

Look for: the setup script reads the three device lists from the environment and sets the disks' owner; the password is decrypted from the mounted Secret and never printed.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

THIS LAB'S OUTPUT

```
00:50:42  oracommon - setasmdiskperm :CRS_ASM_DEVICE_LIST variable is set
00:50:42  … chmod 660 /dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f;chown grid:asmadmin …
00:50:42  oracommon - setasmdiskperm :RECO_ASM_DEVICE_LIST variable is set
00:50:42  … chmod 660 /dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5;chown grid:asmadmin …
00:50:42  oracommon - setasmdiskperm :DB_ASM_DEVICE_LIST variable is set
00:50:42  … chmod 660 /dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222;chown grid:asmadmin …
01:19:51  orasetupenv - setbanner :
          Since OP_TYPE is set to setuprac, setup will be initiated on this node
01:20:02  … openssl pkeyutl -decrypt -pkeyopt "rsa_padding_mode:oaep" … -in "/mnt/.dbsecrets/pwdfile.enc" …
01:20:02  oracommon - setpassword :ORACLE_PWD set to HIDDEN_STRING generated using encrypted password file
```

The status once the install finished

Look for: `OPEN PRIMARY 19.32.0.0.0`, the connect string, `AVAILABLE`.

COMMAND

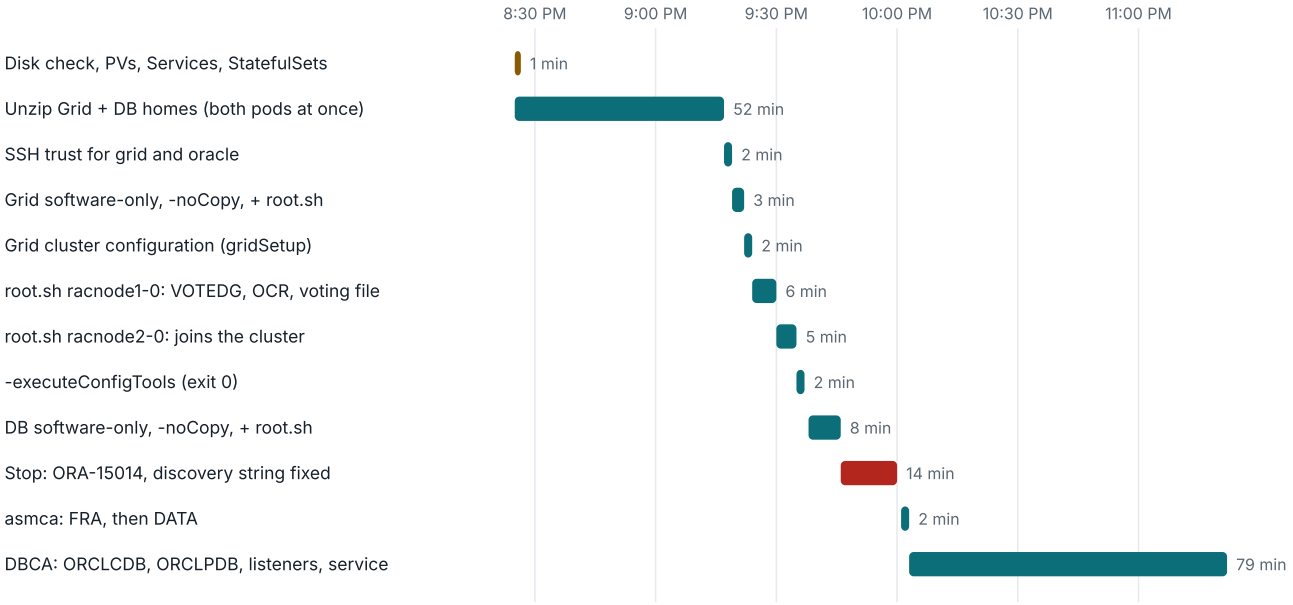
```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac get racdatabase racdb
```

THIS LAB'S OUTPUT

```
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
```

6 The install sequence

The install, step by step: about three hours from apply to ready



Ready: "ORACLE RAC DATABASE IS READY TO USE" at 11:22 PM. Then by hand: second control file and redo members in DATA (a few minutes).

Times from this lab's run (US Eastern). The bars share one time scale; the two long ones are disk-bound.

Each node builds its own homes. The gold-image zips are staged on both workers, so each pod unzips Grid and the database home itself, and every installer runs with `-noCopy`: nothing is copied between nodes. The software-only installs register and relink each home in place; then one cluster configuration run on `racnode1-0` creates the cluster, and `root.sh` on each node starts Clusterware there (VOTEDG is created on the first node).

Where the time goes: the unzips (about 22 GB written by two pods at once) and DBCA. Both are bound by the one physical drive under all the VM disks; the actual Grid and database installs take under 30 minutes together.

The red bar is the one stop in this run: ASM couldn't see the DATA and FRA disks (ORA-15014) because the setup script had guessed a disk pattern from the vote disk's name. Widening the discovery string and rerunning the setup script resumed the install where it stopped; the YAML now sets `CRS_ASM_DISCOVERY_STRING` so a new build doesn't stop there.

See it on the lab

Commands from this lab's build and what they printed (... marks cut text). All are read-only except the `kubectl apply` in section 5, which creates the database.

Following the install: key lines of the setup log on racnode1-0 (UTC; 8:25 PM Eastern = 00:25 UTC)

Look for: the unzips, the SSH trust, Grid software and `root.sh` on both nodes, the DB home, then the stop at ORA-15014.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

THIS LAB'S OUTPUT

```
00:25:51  … su - grid -c " unzip -q /mnt/stage/software/oracle-gridhome-19.32.zip -d /u01/app/19c/grid"
00:50:39  … OS command returned code : 0                        (Grid unzip done, 25 min)
00:50:42  … su - oracle -c " unzip -q /mnt/stage/software/oracle-dbhome-19.32.zip -d
/u01/app/oracle/product/19c/dbhome_1"
01:17:57  orasshsetup - setupsshusekey :Configuring SSH trust racnode1-0 -> racnode1-0
01:20:02  orasetupenv - finalizeorasetupenv :Final SSH verification successful on all cluster nodes
01:20:03  … gridSetup.sh  -ignorePreReq -waitforcompletion  -noCopy  -silent oracle.install.option=CRS_SWONLY …
01:23:14  … Successfully Setup Software.
01:24:46  … su - grid -c "ssh racnode1-0 sudo /u01/app/19c/grid/root.sh"
01:30:48  … returned code : 0
01:30:48  … su - grid -c "ssh racnode2-0 sudo /u01/app/19c/grid/root.sh"
01:35:27  … returned code : 0
01:35:27  … gridSetup.sh -executeConfigTools -responseFile /tmp/grid.rsp -silent
01:38:39  oraracprov - logstep :[RAC] db_sw_install :: start
01:46:45  … Successfully Setup Software.            (both nodes)
01:47:06  ERROR: … ORA-15014: path '/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5' is not in the
discovery set
```

The red bar: widen the discovery string, rerun the setup script, and it resumes

Look for: Grid is skipped as already done; FRA and DATA are created; DBCA starts.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

THIS LAB'S OUTPUT

```
02:00:08  oracommon - checkgiinstalled :
          Grid is already installed on this machine and /etc/oracle also exist. Skipping Grid setup..
02:00:40  oraracprov - cluchecks :All required CVU checks passed successfully on node racnode1-0
02:02:13  … asmca -silent -createDiskGroup -diskGroupName FRA -redundancy EXTERNAL -disk '/dev/disk/by-id/ata-
VBOX_HARDDISK_VB204551ec-809e56f5'
02:02:52  … returned code : 0 … [INFO] [DBT-30001] Disk groups created successfully.
02:02:54  … asmca -silent -createDiskGroup -diskGroupName DATA -redundancy EXTERNAL -disk '/dev/disk/by-id/ata-
VBOX_HARDDISK_VB74717f76-6ef41222'
02:03:31  … returned code : 0 … [INFO] [DBT-30001] Disk groups created successfully.
02:03:32  oraracprov - logstep :[RAC] create_db :: start
```

The end of DBCA: the database is ready

Look for: `ORCLPDB` open on all instances, the service started, then the banner.

COMMAND

```
# op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

THIS LAB'S OUTPUT

```
2026-10-28T03:21:31  Post plug operations are now complete.
                     Pluggable database ORCLPDB with pdb id - 3 is now marked as NEW.
2026-10-28T03:22:12  Pluggable database ORCLPDB opened read write
2026-10-28T03:22:38  Completed: alter pluggable database "ORCLPDB" open  instances=all
2026-10-28T03:22:56  ORCLPDB(3):Started service orclpdbsvc/orclpdbsvc/orclpdbsvc
ORACLE RAC DATABASE IS READY TO USE
```

7 Design choices and limits

CHOICE	WHY	TRADE-OFF
Raw shared disks by `/dev/disk/by-id`	names are stable and identical on both workers; no storage class can share block disks on VirtualBox	one more setting: `CRS_ASM_DISCOVERY_STRING`
Three disk groups (VOTEDG, DATA, FRA)	cluster files apart from the database; the recovery area can fill without stopping the cluster	three disks to manage
Control file and redo in DATA **and** FRA	the setup put both only in FRA; a second copy each in DATA means one disk group can be lost	a few extra writes per commit
Each node unzips its own homes (`-noCopy`)	no 11 GB copy over the network during the install	zips staged on both workers
Operator with one controller replica	leaves enough memory on each worker for a 16 GiB pod	no standby controller (only one is ever active)
16 GiB Guaranteed pods, 4 GiB hugepages, SGA 3 GB	the operator's minimum; the SGA fits in hugepages and SGA + PGA stays within 80 % of the pod	small database, fine for a lab
Clean stop with `crsctl stop crs` on each pod, then the VMs from inside	the database stops with its own stop option (`immediate`) but its target stays `ONLINE`, so it opens by itself after a cold start	a 2-step runbook to stop and a 2-step one to start (guide 14.6); `srvctl stop database` would need a manual start
`SKIP_DBCA=true` added to the env ConfigMaps after creation	a pod restart re-runs Oracle's setup script, which would otherwise try DBCA if an instance isn't open yet	adding it rolls both pods once; the state file stays `provisioning`
Namespace-scoped operator	it only acts in `rac` and its own namespace	extra RBAC for nodes, PVs and Multus

Limits of this design:

- **One physical drive.** All VM disks, shared or not, are files on D:. EXTERNAL redundancy and the control-file copies protect against a lost disk group, not a lost drive, and the drive is the bottleneck (the alert log reports slow redo writes).

- **VIPs can't move.** A VIP is its pod's own IP, so client failover relies on SCAN and services, not on VIP relocation.
- **Access from outside is hand-made.** The operator can publish node listeners as NodePorts (`baseLsnrTargetPort`) only when the database is first created; here the extra listener ports, the explicit `local_listener`, the NodePorts and the Windows forwards were added afterwards (section 3). A new build should use the operator's fields instead.
- **Restarts depend on a guard.** Grid's local files (`/etc/oracle`, the ohasd unit) live in the container; on every start Oracle's setup script restores them with `rootcrs.sh -updateosfiles` and starts Clusterware, which moves the VIPs and SCAN to the pods' new IPs. Without `SKIP_DBCA=true` in the env ConfigMaps the same script would go on to DBCA whenever an instance isn't open yet; with it, a restart of both pods came back without help (lab guide, Phase 13).
- **Stopping the lab:** `crsctl stop crs`**, not** `srvctl stop database`**.** A shutdown and cold start of all three VMs came back without a manual step, in about 12 minutes from VM start to both instances open (lab guide, Phase 14). That needs the database's Clusterware target to stay `ONLINE`: `srvctl stop database` sets it to `OFFLINE`, and the database then stays down after the start. Kubernetes gives a pod only 30 s to stop, too short for RAC, so Clusterware is stopped by hand before the VMs. A hard power-off hasn't been tested.
- **SCAN from outside goes through one pod.** The operator's `racnode-scan-lsnr` Service has one endpoint, racnode1-0's SCAN listener, so Windows can't reach SCAN while racnode1-0 is down, although the other instance is open.
- **Configuration changes restart the whole cluster.** The operator keeps a hash of each env ConfigMap in the pod template, so changing one rolls both RAC pods together, stopped within 30 s with the database open. `envVars` changes don't reach an existing database at all for this spec style.
- **No session failover in thin mode.** `python-oracledb` thin has no TAF: when an instance or its service goes away, a **new** connection lands on the other instance, but an open session doesn't move.
- **Platform.** Oracle documents the RAC controller for Oracle Cloud Native Environment. A kubeadm cluster with CRI-O on VirtualBox works, as this build shows, but is not a supported production platform.

8 Terms used here

TERM	MEANING
RAC	Real Application Clusters: one database opened by several instances on several nodes at once, over shared storage
Grid Infrastructure (CRS, CSS)	Oracle's clusterware: membership (CSS), resource management (CRS), and ASM
ASM, disk group	Oracle's volume manager; a disk group is a set of disks ASM stripes files over
OCR, voting file	the cluster registry, and the file nodes use to decide who stays in the cluster
SCAN	Single Client Access Name: one name for the whole cluster; its listeners redirect each client to the least busy instance
VIP	a node's virtual IP; here the same as its pod IP
CDB, PDB, service	container database, pluggable database inside it, and the name clients connect to
Pod, StatefulSet	a pod is one or more containers on a node; a StatefulSet keeps a pod's name and volumes stable
Headless Service	a DNS name that returns the pod IPs directly, with no load-balancing virtual IP
NodePort Service	opens the same port on every node and forwards it to the selected pods
PV, PVC	a PersistentVolume is a piece of storage; a PersistentVolumeClaim is a pod's request for it
Multus, macvlan, NAD	Multus adds extra pod interfaces; macvlan gives each one its own MAC on a host NIC; a NetworkAttachmentDefinition describes one such network
Operator, CRD, reconcile	an operator adds a custom resource type (CRD) and keeps making the cluster match it
Admission webhook	a check the API server calls before it stores an object

Built and measured on this lab's own run, 27–28 October 2026: Oracle Database Operator v2.2.0, Grid Infrastructure and Database 19.32, Kubernetes v1.36.5.